# Experiment 2: Implement and Demonstrate Best First Search Algorithm on Missionaries-Cannibals Problems using Python

## EVALUATION

| | |
|---|---|
| **Writeup (3M)** | |
| **Execution (5M)** | |
| **Viva Voce (2M)** | |
| **Total Marks (10M)** | |
| **Faculty Signature and Date** | |

## 2.1. AIM

To implement and demonstrate the **Best First Search Algorithm** for solving the **Missionaries and Cannibals Problem** using Python.

## 2.2. PROBLEM DESCRIPTION

The Missionaries and Cannibals problem is a classical Artificial Intelligence search problem.
There are:

- Three missionaries and three cannibals on the left side of a river.
- One boat that can carry a maximum of two people at a time.
- The objective is to transfer all missionaries and cannibals safely to the right side of the river.

**Condition**
At no point on either side of the river should the number of cannibals exceed the number of missionaries; otherwise, the missionaries will be eaten.

Each state can be represented as:

(M_left, C_left, Boat_Position)

where:

- M_left = number of missionaries on the left bank
- C_left = number of cannibals on the left bank
- Boat_Position = 1 (left bank), 0 (right bank)

Initial State:
(3, 3, 1)

Goal State:
(0, 0, 0)

## 2.3. CONSTRAINTS AND ASSUMPTIONS

**Constraints**

1. Boat capacity is limited to two persons.

2. At least one person must be in the boat for crossing (row the boat).
3. The number of cannibals must never exceed missionaries on either bank unless missionaries are zero.
4. Only valid and safe states are allowed.
5. Repeated states should not be revisited.

**Assumptions**

1. All missionaries and cannibals are identical.
2. Crossing time is ignored.
3. River width and direction do not affect the problem.
4. Initial configuration is (3,3,1).

## 2.4. ALGORITHM

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
     if EMPTY?(frontier) then return failure
     node ← POP(frontier)  /* chooses the shallowest node in frontier */
     add node.STATE to explored
     for each action in problem.ACTIONS(node.STATE) do
        child ← CHILD-NODE(problem, node, action)
        if child.STATE is not in explored or frontier then
           if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
           frontier ← INSERT(child, frontier)
```

**Figure 3.11** Breadth-first search on a graph.

State Representation

$$(M_L, C_L, B)$$

where

$M_L = Missionaries\ on\ Left\ Bank$

$C_L = Cannibals\ on\ Left\ Bank$

$B = Boat\ Position\ (Left\ or\ Right)$

Initial State

(3,3,Left)

Goal State

(0,0, Right)

Actions

(1M), (2M),(1C),(2C),(1M,1C)

Validity Condition

$$0 \leq M \leq 3$$

$$0 \leq C \leq 3$$

$$M_L \geq C_L \ or \ M_L = 0$$

$$M_R \geq C_R \ or \ M_R = 0$$

## 2.5. PROGRAM

https://github.com/mayurjambit/BAD402

## 2.6. OUTPUT